

Project Title: AI FAQ Assistant API

Team ID: SWTID-2026-5032

Team Leader:

MOGANASANKARI.S & moganasankari8@gmail.com

Team Member:

MANIMEGALAI.S & manimegalaisakthvel54@gmail.com

PAVITHRA.M & pavithra43845@gmail.com

POORNIMA.S & poorni0033@gmail.com

ABSTRACT:

AnswerBot is an AI-powered FAQ and customer-support automation backend. It is a REST API built with Node.js and Express.js, using MongoDB (via Mongoose) for storage, JWT and bcrypt for secure authentication, and Google Gemini AI to generate FAQ questions and answers from a topic.

Sample user: Mogan | `mogan@example.com` | `Mogan@123`

INTRODUCTION

Support teams answer the same questions repeatedly, and maintaining a large FAQ by hand is slow. AnswerBot provides a headless backend where users register, log in, create and search FAQs, and generate FAQ content with AI. Public routes (view/search) are separated from protected routes (create/update/delete/AI).

PROBLEM STATEMENT

- Manual creation and upkeep of FAQ content is time-consuming.
- FAQs are hard to categorize and search by keyword.
- Private FAQ records need secure access control.
- There is no simple way to auto-generate FAQ content.

OBJECTIVES

- Secure registration and login (bcrypt + JWT).
- CRUD, category organization and keyword search for FAQs.
- AI-based FAQ generation with Google Gemini.
- Schema validation and standardized error responses.

SYSTEM OVERVIEW

Flow: Client → Express Server → Auth Middleware → Controller/Service → MongoDB

AI flow: Client → Express Server → AI Service (Gemini) → Generated FAQ → Response

SOFTWARE REQUIREMENTS

Software	Purpose
Node.js 18+ / npm 9+	Runtime and package manager
Express.js	REST routing
MongoDB + Mongoose	Database and schemas
JWT, bcrypt	Authentication and password hashing
@google/genai	Gemini AI integration
Postman / Thunder Client	API testing
VS Code	Code editor

Setup Commands

```
npm init -y
npm install express mongoose dotenv cors jsonwebtoken bcrypt @google/genai
npm install --save-dev nodemon
```

package.json (scripts)

```
"scripts": {
```

```
"start": "node src/server.js",  
"dev": "nodemon src/server.js"  
}
```

.env

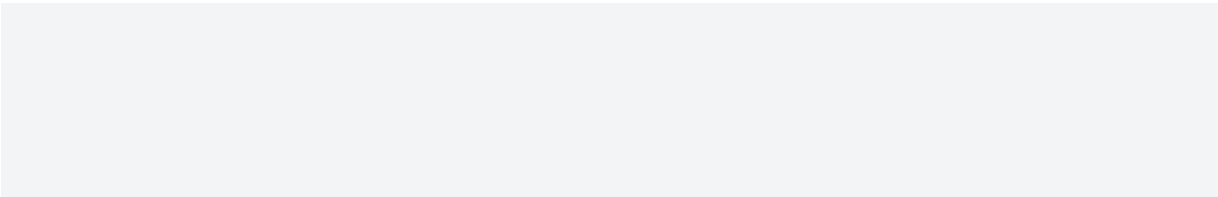
```
PORT=5000  
MONGO_URI=mongodb://127.0.0.1:27017/answerbot  
JWT_SECRET=change_this_secret  
JWT_EXPIRES_IN=1d  
GEMINI_API_KEY=your_gemini_api_key
```

HARDWARE REQUIREMENTS

Component	Requirement
Processor	Intel Core i5 (8th Gen) / AMD Ryzen 5 or above
RAM	8 GB minimum (16 GB recommended)
Storage	1 GB free space
Internet	Needed for cloud DB and Gemini API

SYSTEM ARCHITECTURE

The project follows a layered MVC-style architecture: Route → Middleware → Controller → Service → Model → Database, with an AI integration layer for Gemini.



answerbot/

```
|— .env
|— package.json
└— src/
    |— server.js
    |— app.js
    |— config/db.js
    |— middleware/
    |   |— authMiddleware.js
    |   └— errorMiddleware.js
    |— models/
    |   |— User.js
    |   └— FAQ.js
    |— controllers/
    |   |— authController.js
    |   └— faqController.js
    |— services/aiService.js
    └— routes/
        |— authRoutes.js
        |— faqRoutes.js
        └— aiRoutes.js
```

1.DATABASE DESIGN

One user can create many FAQs (FAQ.createdBy → User._id); a category groups many FAQs.

src/config/db.js

```
const mongoose = require('mongoose');
```

```
module.exports = async () => {  
  await mongoose.connect(process.env.MONGO_URI);  
  console.log('MongoDB connected');  
};
```

src/models/User.js

```
const mongoose = require('mongoose');  
const bcrypt = require('bcrypt');  
  
const userSchema = new mongoose.Schema({  
  name: { type: String, required: true, trim: true },  
  email: { type: String, required: true, unique: true, lowercase: true },  
  password: { type: String, required: true, minlength: 6, select: false },  
  role: { type: String, enum: ['admin', 'user'], default: 'user' }  
}, { timestamps: true });  
  
userSchema.pre('save', async function (next) {  
  if (!this.isModified('password')) return next();  
  this.password = await bcrypt.hash(this.password, 10);  
  next();  
});  
  
userSchema.methods.matchPassword = function (plain) {  
  return bcrypt.compare(plain, this.password);  
};  
  
module.exports = mongoose.model('User', userSchema);
```

src/models/FAQ.js

```
const mongoose = require('mongoose');

const faqSchema = new mongoose.Schema({
  question: { type: String, required: true, trim: true },
  answer: { type: String, required: true },
  category: { type: String, default: 'General', trim: true },
  createdBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true }
}, { timestamps: true });

faqSchema.index({ question: 'text', answer: 'text', category: 'text' });

module.exports = mongoose.model('FAQ', faqSchema);
```

MODULES AND SOURCE CODE

Module	Description
Authentication	Register/login with bcrypt and JWT
FAQ Management	CRUD operations with ownership checks
Search	Keyword search: GET /api/faqs/search?q=configure
Category	Group FAQs using the category field
AI Generation	Generate FAQs from a topic using Gemini
Error Handling	Centralized JSON error responses

2.Server and App

src/server.js

```
require('dotenv').config();
const app = require('./app');
const connectDB = require('./config/db');

const PORT = process.env.PORT || 5000;

connectDB()
  .then(() => app.listen(PORT, () => console.log(`Server running on port ${PORT}`)))
  .catch(err => { console.error(err.message); process.exit(1); });
```

src/app.js

```
const express = require('express');
const cors = require('cors');
const { notFound, errorHandler } = require('./middleware/errorMiddleware');

const app = express();
app.use(cors());
app.use(express.json());

app.get('/', (req, res) => res.json({ message: 'AnswerBot API running' }));
app.use('/api/auth', require('./routes/authRoutes'));
app.use('/api/faqs', require('./routes/faqRoutes'));
app.use('/api/ai', require('./routes/aiRoutes'));

app.use(notFound);
app.use(errorHandler);
```

```
module.exports = app;
```

3.Authentication Module

src/controllers/authController.js

```
const jwt = require('jsonwebtoken');
const User = require('../models/User');

const signToken = id =>
  jwt.sign({ id }, process.env.JWT_SECRET, { expiresIn:
process.env.JWT_EXPIRES_IN });

exports.register = async (req, res, next) => {
  try {
    const { name, email, password } = req.body;
    if (await User.findOne({ email }))
      return res.status(400).json({ success: false, message: 'Email already
registered' });

    const user = await User.create({ name, email, password });
    res.status(201).json({
      success: true,
      token: signToken(user._id),
      user: { id: user._id, name: user.name, email: user.email, role: user.role }
    });
  } catch (err) { next(err); }
};

exports.login = async (req, res, next) => {
```

```

try {
  const { email, password } = req.body;
  const user = await User.findOne({ email }).select('+password');
  if (!user || !(await user.matchPassword(password)))
    return res.status(401).json({ success: false, message: 'Invalid email or
password' });

  res.json({
    success: true,
    token: signToken(user._id),
    user: { id: user._id, name: user.name, email: user.email, role: user.role }
  });
} catch (err) { next(err); }
};

```

src/middleware/authMiddleware.js

```

const jwt = require('jsonwebtoken');
const User = require('../models/User');

exports.protect = async (req, res, next) => {
  try {
    const header = req.headers.authorization || '';
    if (!header.startsWith('Bearer '))
      return res.status(401).json({ success: false, message: 'Not authorized, token
missing' });

    const decoded = jwt.verify(header.split(' ')[1], process.env.JWT_SECRET);
    req.user = await User.findById(decoded.id);
    if (!req.user)

```

```

    return res.status(401).json({ success: false, message: 'User no longer exists'
    });
    next();
  } catch (err) {
    res.status(401).json({ success: false, message: 'Invalid or expired token' });
  }
};

exports.authorize = (...roles) => (req, res, next) =>
  roles.includes(req.user.role)
    ? next()
    : res.status(403).json({ success: false, message: 'Access denied' });

```

src/routes/authRoutes.js

```

const router = require('express').Router();
const { register, login } = require('../controllers/authController');

router.post('/register', register);
router.post('/login', login);

module.exports = router;

```

4.FAQ, Search and Category Module

src/controllers/faqController.js

```

const FAQ = require('../models/FAQ');

exports.createFAQ = async (req, res, next) => {
  try {

```

```

    const { question, answer, category } = req.body;
    const faq = await FAQ.create({ question, answer, category, createdBy:
req.user._id });
    res.status(201).json({ success: true, data: faq });
  } catch (err) { next(err); }
};

exports.getFAQs = async (req, res, next) => {
  try {
    const filter = req.query.category ? { category: req.query.category } : {};
    const faqs = await FAQ.find(filter).sort('-createdAt');
    res.json({ success: true, count: faqs.length, data: faqs });
  } catch (err) { next(err); }
};

exports.searchFAQs = async (req, res, next) => {
  try {
    const q = (req.query.q || "").trim();
    if (!q) return res.status(400).json({ success: false, message: 'Query "q" is
required' });

    const regex = new RegExp(q.replace(/[*+?^$ {}()[\]\]/g, '\\$&'), 'i');
    const faqs = await FAQ.find({
      $or: [{ question: regex }, { answer: regex }, { category: regex }]
    });
    res.json({ success: true, count: faqs.length, data: faqs });
  } catch (err) { next(err); }
};

exports.updateFAQ = async (req, res, next) => {

```

```
try {
  const faq = await FAQ.findById(req.params.id);
  if (!faq) return res.status(404).json({ success: false, message: 'FAQ not found'
});
  if (faq.createdBy.toString() !== req.user._id.toString() && req.user.role !==
'admin')
    return res.status(403).json({ success: false, message: 'Not allowed' });

  Object.assign(faq, req.body);
  await faq.save();
  res.json({ success: true, data: faq });
} catch (err) { next(err); }
};

exports.deleteFAQ = async (req, res, next) => {
  try {
    const faq = await FAQ.findById(req.params.id);
    if (!faq) return res.status(404).json({ success: false, message: 'FAQ not found'
});
    if (faq.createdBy.toString() !== req.user._id.toString() && req.user.role !==
'admin')
      return res.status(403).json({ success: false, message: 'Not allowed' });

    await faq.deleteOne();
    res.json({ success: true, message: 'FAQ deleted' });
  } catch (err) { next(err); }
};
```

src/routes/faqRoutes.js

```
const router = require('express').Router();
const { protect } = require('../middleware/authMiddleware');
const c = require('../controllers/faqController');

router.get('/search', c.searchFAQs);    // public (before /:id)
router.get('/', c.getFAQs);           // public
router.post('/', protect, c.createFAQ); // private
router.put('/:id', protect, c.updateFAQ); // private
router.delete('/:id', protect, c.deleteFAQ); // private

module.exports = router;
```

5.AI Generation Module

src/services/aiService.js

```
const { GoogleGenAI } = require('@google/genai');
const ai = new GoogleGenAI({ apiKey: process.env.GEMINI_API_KEY });

exports.generateFAQ = async topic => {
  const prompt = `Create one FAQ about "${topic}". Reply ONLY with JSON:
{"question": "...", "answer": "...", "category": "..."}`;

  const response = await ai.models.generateContent({
    model: 'gemini-2.5-flash',
    contents: prompt
  });

  const text = response.text.replace(/``json|``/g, "").trim();
  return JSON.parse(text);
```

```
};
```

src/routes/aiRoutes.js

```
const router = require('express').Router();
const { protect } = require('../middleware/authMiddleware');
const FAQ = require('../models/FAQ');
const { generateFAQ } = require('../services/aiService');

router.post('/generate-faq', protect, async (req, res, next) => {
  try {
    const { topic, save } = req.body;
    if (!topic) return res.status(400).json({ success: false, message: 'Topic is required' });

    const generated = await generateFAQ(topic);
    if (save) generated._saved = await FAQ.create({ ...generated, createdBy: req.user._id });

    res.json({ success: true, data: generated });
  } catch (err) { next(err); }
});

module.exports = router;
```

Error Handling Module

src/middleware/errorMiddleware.js

```
exports.notFound = (req, res, next) => {
  res.status(404);
```

```

    next(new Error(`Route not found: ${req.originalUrl}`));
  };

exports.errorHandler = (err, req, res, next) => {
  let status = res.statusCode === 200 ? 500 : res.statusCode;
  let message = err.message;

  if (err.name === 'ValidationError') {
    status = 400;
    message = Object.values(err.errors).map(e => e.message).join(', ');
  }
  if (err.code === 11000) { status = 400; message = 'Duplicate value entered'; }

  res.status(status).json({
    success: false,
    message,
    stack: process.env.NODE_ENV === 'production' ? undefined : err.stack
  });
};

```

USER ROLES

Role	Permissions
Admin	Manage users, categories and all FAQs; monitor AI content
Content Creator	Create, edit, delete own FAQs; generate FAQs with AI
Authenticated User	View and search FAQs; generate AI-assisted answers; view profile

Public User	View and search published FAQs only
-------------	-------------------------------------

API ENDPOINTS

Method	Endpoint	Access	Description
POST	/api/auth/register	Public	Register user
POST	/api/auth/login	Public	Login, returns JWT
GET	/api/faqs	Public	List FAQs
GET	/api/faqs/search?q=configure	Public	Search FAQs
POST	/api/faqs	Private	Create FAQ
PUT	/api/faqs/:id	Private	Update FAQ
DELETE	/api/faqs/:id	Private	Delete FAQ
POST	/api/ai/generate-faq	Private	Generate FAQ with AI

Sample Requests and Responses

POST /api/auth/register

```
{
  "name": "Mogan",
  "email": "mogan@example.com",
  "password": "Mogan@123"
}
```

Response 201

```
{
  "success": true,
```

```
"token": "eyJhbGciOiJIUzI1NiIs...",
"user": { "id": "665f...", "name": "Mogan", "email": "mogan@example.com",
"role": "user" }
}
```

POST /api/faqs (Authorization: Bearer <token>)

```
{
  "question": "How do I configure the application?",
  "answer": "Set the values in the .env file and restart the server.",
  "category": "Setup"
}
```

POST /api/ai/generate-faq (Authorization: Bearer <token>)

```
{ "topic": "Resetting a password", "save": true }
```

TESTING

Test with Postman, Thunder Client or the curl commands below. Protected routes must reject requests without a valid token.

1. Register

```
curl -X POST http://localhost:5000/api/auth/register \
  -H "Content-Type: application/json" \
  -d
'{"name":"Mogan","email":"mogan@example.com","password":"Mogan@123"}'
```

2. Login (copy the token from the response)

```
curl -X POST http://localhost:5000/api/auth/login \
  -H "Content-Type: application/json" \
```

```
-d '{"email":"mogan@example.com","password":"Mogan@123"}'
```

3. Create FAQ

```
curl -X POST http://localhost:5000/api/faqs \  
-H "Authorization: Bearer <TOKEN>" -H "Content-Type: application/json" \  
-d '{"question":"How to configure?","answer":"Edit .env","category":"Setup"}'
```

4. Search FAQ

```
curl "http://localhost:5000/api/faqs/search?q=configure"
```

5. Generate FAQ with AI

```
curl -X POST http://localhost:5000/api/ai/generate-faq \  
-H "Authorization: Bearer <TOKEN>" -H "Content-Type: application/json" \  
-d '{"topic":"Resetting a password"}'
```

6. Unauthorized check (expect 401)

```
curl -X POST http://localhost:5000/api/faqs -H "Content-Type: application/json" \  
-d '{}'
```

Test Case	Expected	Result
Register new user	201	Token and user returned
Login with valid credentials	200	JWT returned
Login with wrong password	401	Error message
Create FAQ without token	401	Access rejected
Create FAQ with token	201	FAQ stored
Search "configure"	200	Matching FAQs
AI generate FAQ	200	Question, answer, category

CONCLUSION

AnswerBot combines Node.js, Express.js, MongoDB, JWT, bcrypt and Google Gemini into a modular backend that simplifies FAQ management. Users can securely register, manage and search FAQs, while the AI module generates content from a topic. The REST design allows easy integration with future web and mobile applications.

Sample credentials: Mogan | mogan@example.com | Mogan@123